



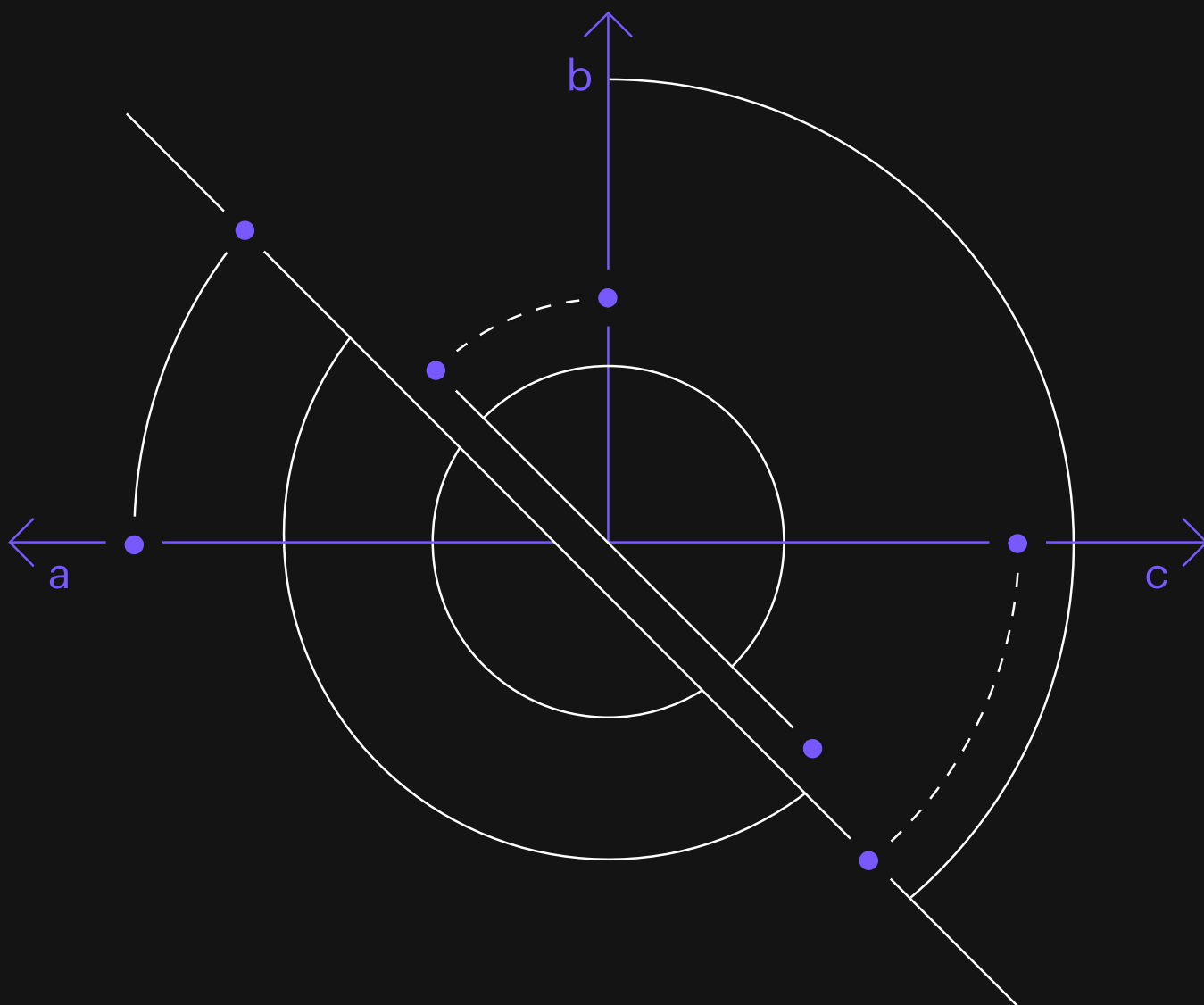
Машинное обучение

Лонгрид

Глубокое обучение

03

Операция свёртки



Содержание

1. Зачем нужна ещё одна архитектура	3
1.1. Полносвязная сеть — универсальный аппроксиматор, и этого мало	3
1.2. Две идеи: локальность и разделение весов	4
2. Операция свёртки	6
2.1. Базовая свёртка	6
2.2. Параметры свёрточного слоя	7
2.2.1. Padding	7
2.2.2. Stride	7
2.2.3. Dilation	8
2.3. Трёхмерный вход: каналы	9
2.4. Свёртка 1×1	10
2.5. Виды свёрток и экономия параметров	10
2.5.1. Разделимая по осям свёртка (spatial separable)	10
2.5.2. Сгруппированная свёртка (group convolution)	11
2.5.3. Depthwise и depthwise separable	11
2.6. Транспонированная свёртка	12
2.6.1. Почему именно так и почему транспонированная	13
3. Обратный проход через свёрточный слой	14
3.1. Свёртка — это линейная операция	14
3.2. Градиенты в матричной форме	15
3.3. Градиент по входу — это свёртка с развёрнутым ядром	15
3.4. Градиент по фильтру — это свёртка входа с картой градиентов	16
4. Пулинг	18
4.1. Что это	18
4.2. Зачем: инвариантность к малым сдвигам	18
4.3. Как через пулинг идёт градиент	18
5. Рецептивное поле	19
6. Dropout и Batch Normalization в свёрточных слоях	19
7. Итоги	20

Свёрточный слой изнутри: параметры, каналы, градиенты, пулинг и рецептивное поле

О чём эта лекция

До сих пор в курсе была одна базовая архитектура — многослойный персептрон. Здесь появляется второй строительный блок — свёрточный слой. Разбираются сама операция и все её параметры (padding, stride, dilation), трёхмерный вход с каналами, свёртка 1×1 , экономные виды свёрток, транспонированная свёртка, полный вывод обратного прохода, пулинг, рецептивное поле и то, как в свёрточных слоях устроены Dropout и Batch Normalization.

Граница конспекта

Здесь разбирается **операция**: из чего состоит свёрточная сеть. Как из этих блоков собирались знаменитые архитектуры от LeNet до ConvNeXt и как пользоваться предобученными сетями — [лекция 6](#), инициализация, нормализация и Dropout в общем виде — [лекция 3](#), оптимизаторы — [лекция 4](#). В этой лекции они упоминаются только там, где у них есть свёрточная специфика.

Про анимации в этом файле

Это версия конспекта с живыми анимациями: у пяти рисунков есть кнопки воспроизведения. Анимации в PDF работают в настольных **Adobe Acrobat Reader**, **PDF-XChange** и **Foxit**. В браузере, на телефоне и в предпросмотре Overleaf будет виден только первый кадр. Содержание при этом не теряется, все ключевые состояния продублированы статичными рисунками рядом.

1. Зачем нужна ещё одна архитектура

1.1. ПОЛНОСВЯЗНАЯ СЕТЬ — УНИВЕРСАЛЬНЫЙ АППРОКСИМАТОР, И ЭТОГО МАЛО

Формально возражений к многослойному персептрону нет: теорема об универсальной аппроксимации говорит, что достаточно широкая сеть с одним скрытым слоем приближает любую непрерывную функцию на компакте со сколь угодно малой ошибкой. Казалось бы, вопрос закрыт.

Проблема не в выразительности, а в цене. Посмотрим, во что обходится полносвязный слой на изображении. Возьмём небольшую картинку 300×300 в градациях серого. Для полносвязного слоя её надо вытянуть в вектор:

$$300 \times 300 = 90\,000 \text{ входов.}$$

Пусть первый слой сохраняет размерность, то есть тоже выдаёт 90 000 чисел. Полносвязный слой связывает каждый вход с каждым выходом, поэтому число параметров равно

$$90\,000 \times 90\,000 = 8,1 \times 10^9.$$

Восемь миллиардов параметров — в одном слое на маленькой чёрно-белой картинке. Во float32 это примерно 32 ГБ только на веса, не считая градиентов и состояния оптимизатора. Для цветной картинки входов втрое больше и число параметров растёт в девять раз.

СЛЕДСТВИЕ

Три следствия сразу

- **Память.** Веса такого слоя не помещаются в видеопамять; про хранение моментов Adam, которые удваивают или утраивают требования, речи вообще не идёт.
- **Время.** Каждый шаг — матрично-векторное умножение с матрицей на 8×10^9 элементов.
- **Переобучение.** Число параметров на порядки больше числа объектов в любой реалистичной выборке.

Значит, нужна архитектура, в которой параметров существенно меньше, но выразительности хватает. Именно это и реализовано в свёрточных сетях. И достигается это двумя ограничениями, которые накладываются на слой.

1.2. ДВЕ ИДЕИ: ЛОКАЛЬНОСТЬ И РАЗДЕЛЕНИЕ ВЕСОВ

Локальность. Полносвязный слой связывает каждый выход со всеми входами. Но для изображения это избыточно: чтобы понять, есть ли в точке край, достаточно посмотреть на окрестность 3×3 пикселя, а не на всю картинку. Ограничим связи небольшим окном, и число параметров упадёт с $HW \times HW$ до $HW \times k^2$.

Разделение весов. Детектор вертикального края в левом верхнем углу и в правом нижнем — это одна и та же операция. Нет смысла учить её дважды. Заставим все позиции использовать один и тот же набор коэффициентов, и параметров останется ровно k^2 на один фильтр, независимо от размера картинки.

Вместе эти два ограничения превращают слой из $8,1 \times 10^9$ параметров в слой из девяти. Разумеется, одного фильтра мало, поэтому на слое их держат десятки или сотни, но порядок величины меняется радикально.



Откуда взялась сама идея

Свёртка с фильтром — это не изобретение глубокого обучения. В классическом компьютерном зрении с 1960-х годов изображение сворачивали с небольшими ядрами вручную подобранных коэффициентов: фильтр Собеля для выделения краёв, гауссово ядро для размытия, лапласиан для резкости и т. д. Например, ядро

$$S_x = \begin{pmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{pmatrix}$$

смотрит на разность интенсивностей в окне 3×3 : если слева и справа яркость заметно разная, отклик большой, значит, здесь граница.

Пощупать руками, как разные ядра действуют на живое изображение, можно в интерактивной демонстрации: setosa.io/ev/image-kernels.

Таких фильтров было придумано много, и подбирали их люди. Идея свёрточной сети ровно одна: **пусть коэффициенты фильтра будут обычными обучаемыми параметрами**, такими же, как веса линейного слоя, и настраиваются градиентным спуском под конкретную задачу.



Исходное изображение



Изображение после применения фильтра

Рисунок 1. Фильтр Собеля на реальном кадре: слева — исходное изображение, справа — отклик фильтра, выделяющего края

2. Операция свёртки



Обозначения

- I — входное изображение высоты H и ширины W ; у трёхмерного входа есть ещё число каналов C ;
- K — ядро (фильтр) размера $h_K \times w_K$; для квадратного ядра пишем просто k . Здесь k — это всегда **размер ядра**, а не номер итерации, как в [лекции 4](#);
- F — число фильтров свёрточного слоя;
- p, s, d — padding, stride и dilation; что это такое, разобрано чуть ниже;
- в разделе про обратный проход X — вход слоя, Y — его выход, $G = \partial L / \partial Y$ — карта градиентов функции потерь L , $\mathcal{K}$ — матрица свёртки, действующая на вытянутый в вектор вход.

2.1. БАЗОВАЯ СВЁРТКА

Пусть есть входное изображение I и небольшое ядро (фильтр) K . Ядро подставляется в каждую возможную позицию внутри изображения. В каждой позиции берётся скалярное произведение ядра с накрытым им фрагментом картинки и результат становится одним элементом выхода.

Для двумерного случая

$$\tilde{I}(i, j) = \sum_{u=0}^{h_K-1} \sum_{v=0}^{w_K-1} K(u, v) I(i + u, j + v), \quad (1)$$

где u пробегает от 0 до $h_K - 1$, v — от 0 до $w_K - 1$, а i и j — все позиции, при которых окно целиком помещается внутри изображения.

Берём окно размера $h_K \times w_K$ с верхним левым углом в позиции (i, j) , скалярно умножаем его на ядро, записываем число. И так для всех (i, j) .

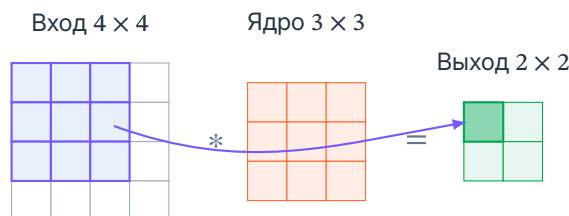


Рисунок 2. Базовая свёртка. Ядро подставляется в каждую из четырёх допустимых позиций; подсвечена первая.

Изображение 4×4 с ядром 3×3 даёт выход 2×2

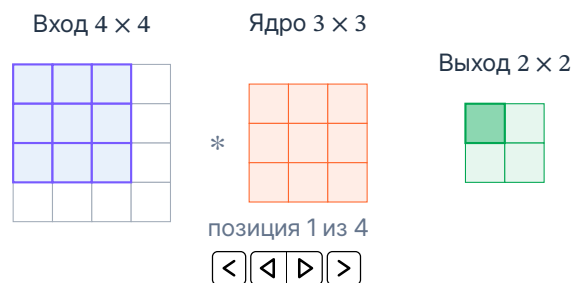


Рисунок 3. Анимация (Acrobat): ядро проходит все четыре позиции, каждая даёт свой элемент выхода

ЗАМЕЧАНИЕ**Свёртка или корреляция**

Строго говоря, формула (1) — это **кросс-корреляция**, а не свёртка в математическом смысле: в настоящей свёртке ядро переворачивается и стоит $I(i - u, j - v)$. В глубоком обучении переворот опускают: коэффициенты ядра всё равно обучаемые, перевёрнутое ядро выучится ровно так же. Термин при этом остался прежним, и слои называются *Conv2d*.

Помнить об этом стоит по одной причине: при выводе обратного прохода переворот ядра появится сам собой, и это не ошибка, а именно та транспонированная операция, которую мы опустили на прямом проходе.

2.2. ПАРАМЕТРЫ СВЁРТОЧНОГО СЛОЯ

У свёртки в том виде, как она реализована в PyTorch, есть три параметра, меняющих геометрию: `padding`, `stride` и `dilation`.

2.2.1. Padding

Идея — добавить по краям входного изображения несколько дополнительных пикселей и только потом применять свёртку. Основное назначение — добиться того, чтобы размер выхода совпадал с размером входа. Без `padding` изображение 5×5 после ядра 3×3 превращается в 3×3 ; если добавить рамку шириной один пиксель, на выходе снова получится 5×5 .

Заполнять рамку можно по-разному: нулями, средним значением, зеркальным отражением края, повторением края — всё это задаётся параметрами. На практике почти всегда используют нули, и это логично: входные данные обычно нормализованы так, что среднее по каналу равно нулю, поэтому добивать нулями — это и есть добивать средним.

2.2.2. Stride

Ядро подставляется не в каждую возможную позицию, а через одну, через две и т. д. При `stride = 2` выход по каждой пространственной оси уменьшается примерно вдвое. Это одновременно и способ сокращать размерность, и способ считать быстрее: позиций, в которых надо вычислять скалярное произведение, становится вчетверо меньше.

Есть и третья причина. Соседние окна свёртки перекрываются на $k - 1$ пикселей, поэтому соседние выходы почти совпадают, подряд идущие отсчёты сильно коррелированы. Шаг прореживает их, оставляя менее избыточное представление.

2.2.3. Dilation

Этот параметр ещё называют **atrous convolution**, от французского *'à trous* — «с дырками». Ядро 3×3 применяется не к девяти соседним пикселям, а к девяти пикселям, расставленным с промежутками.

Смысл в следующем. Любой фильтр смотрит на картинку локально: есть большое изображение, а фильтр оценивает что-то в маленькой окрестности 3×3 . Раздвинув элементы ядра, мы увеличиваем эффективное окно обзора до 5×5 , то есть смотрим на картинку в большем масштабе, **не меняя числа параметров**: было девять коэффициентов, девять и осталось. Полноценный фильтр 5×5 стоил бы 25 параметров.

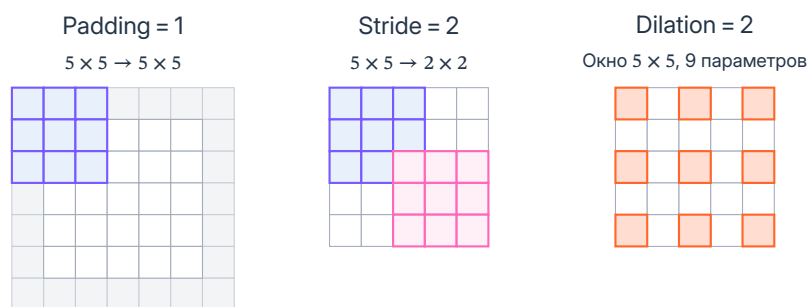


Рисунок 4. Три параметра свёртки. Padding расширяет вход рамкой, stride прореживает позиции ядра, dilation раздвигает само ядро

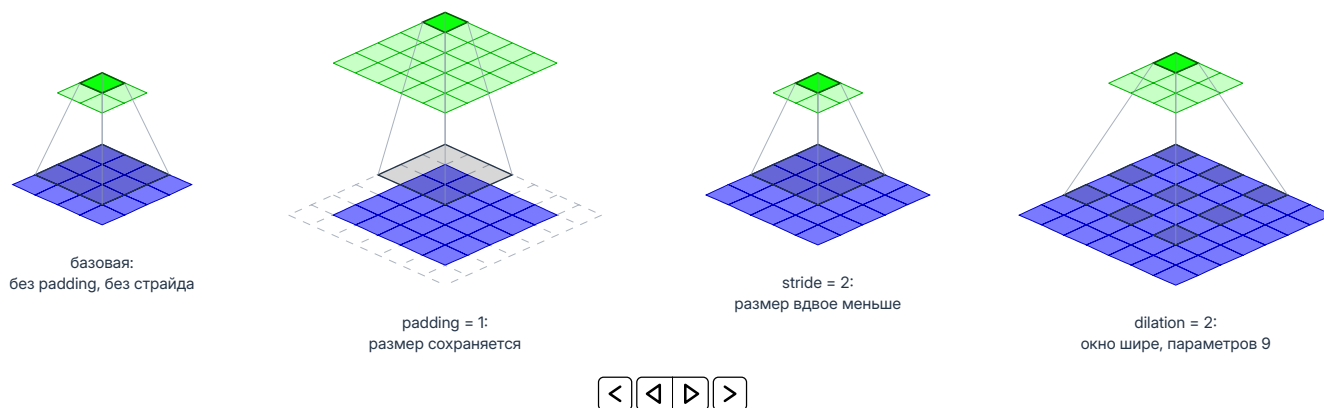


Рисунок 5. Анимация (Acrobat) нарисована векторно: те же четыре режима в движении. Синее — вход, пунктир — нули, серое окно — ядро, зелёное — выход; тёмная клетка вычисляется сейчас, средне-зелёные уже посчитаны. У панелей разная длина цикла, поэтому они движутся каждая в своём ритме



Размер выхода

Все три параметра сводятся в одну формулу. Для входа высоты H , ядра высоты k , padding p , stride s и dilation d

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - d(k - 1) - 1}{s} \right\rfloor + 1. \quad (2)$$

По ширине — ровно так же. Полезно проверить на частных случаях: при $p = 0$, $s = 1$, $d = 1$ получаем $H - k + 1$ (базовая свёртка); при $k = 3$, $p = 1$, $s = 1$, $d = 1$ получаем ровно H (размер сохраняется); при $d = 2$, $k = 3$ эффективный размер ядра равен $d(k - 1) + 1 = 5$.

2.3. ТРЁХМЕРНЫЙ ВХОД: КАНАЛЫ

До сих пор картинка была плоской. На практике на вход свёрточной сети подаётся трёхмерный тензор: высота H , ширина W и число каналов C для RGB-изображения $C = 3$.

Ядро в этом случае тоже трёхмерное: $h_K \times w_K \times C$. Обрати внимание на третью размерность: она равна *полному* числу каналов входа. Такое ядро подставляется во все возможные позиции, но двигается только по двум пространственным осям; в каждой позиции берётся скалярное произведение по всем трём размерностям сразу. Результат — *плоская* карта $H_{\text{out}} \times W_{\text{out}}$.

Одной карты мало, поэтому на слое держат F разных фильтров одинакового размера. Каждый даёт свою плоскую карту, карты складываются в стопку, и выход слоя имеет размер

$$H_{\text{out}} \times W_{\text{out}} \times F.$$

На следующем слое эти F карт играют роль каналов, и всё повторяется. Число параметров одного свёрточного слоя равно

$$\underbrace{h_K \cdot w_K \cdot C \cdot F}_{\text{Один фильтр}} + \underbrace{F}_{\text{Смещения}}. \quad (3)$$

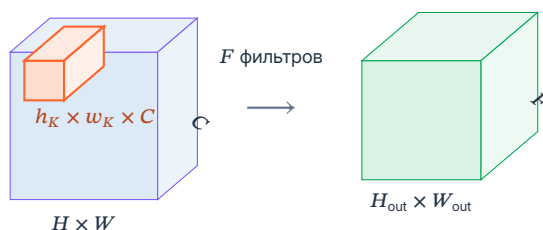


Рисунок 6. Свёрточный слой. Фильтр трёхмерный и по третьей оси накрывает все каналы входа; каждый из F фильтров даёт одну плоскую карту, карты складываются в стопку

**Вопрос: почему по третьей оси берут сразу все каналы?**

Формально ничто не мешает взять ядро глубины меньше C и двигать его ещё и по третьей оси, получая на выходе трёхмерный объект. Почему так не делают?

Ответ в том, что **в пространстве каналов нет отношения соседства**. По пространственным осям оно есть: пиксель имеет соседей, и именно на разности соседних пикселей строится детектор края. В этом весь смысл локального окна. А про каналы нельзя сказать, что красный ближе к зелёному, чем к синему: это просто группа равноправных величин. Скользить окном по такой оси бессмысленно: результат зависел бы от произвольного порядка каналов.

То же и на следующих слоях: там третья размерность — это номера фильтров предыдущего слоя, между которыми тоже нет никакого естественного порядка. Поэтому по третьей оси не скользят, а суммируют сразу всё с обучаемыми коэффициентами.

2.4. СВЁРТКА 1×1

Частный, но очень полезный случай — ядро размера $1 \times 1 \times C$. Пространственно оно ничего не смешивает: смотрит ровно на один пиксель. Зато по каналам оно берёт линейную комбинацию всех C значений. Если таких фильтров взять F штук, слой превращает тензор $H \times W \times C$ в тензор $H \times W \times F$, то есть **меняет число каналов, не трогая пространственный размер**.

Стоит это $C \times F$ параметров и очень дёшево по вычислениям. В следующей лекции мы увидим, что на этом построен весь Inception-блок: перед дорогой свёрткой 3×3 или 5×5 ставят 1×1 , чтобы сбить число каналов, а после — ещё одну, чтобы вернуть обратно.

2.5. ВИДЫ СВЁРТОК И ЭКОНОМИЯ ПАРАМЕТРОВ

По формуле (3) обычный свёрточный слой стоит $h_K w_K \times C \times F$ параметров и почти столько же множителей приходится на каждую позицию выхода. Есть три стандартных способа сделать дешевле; все три разбирают полную свёртку на более простые множители.

2.5.1. Разделимая по осям свёртка (spatial separable)

Если матрица ядра имеет ранг 1, она раскладывается в произведение столбца на строку: $K = uv^T$. Тогда свёртка с ядром $k \times k$ равна каскаду из двух свёрток, $k \times 1$ и $1 \times k$: $2k$ параметров вместо k^2 , и считается быстрее.

Знакомый пример: фильтр Собеля из начала лекции — в точности ранга 1.

$$S_x = \begin{pmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} \begin{pmatrix} -1 & 0 & +1 \end{pmatrix}.$$

Произвольное обучаемое ядро, конечно, ранга 1 не будет, поэтому в сетях факторизацию закладывают заранее, ставя пары $1 \times n$ и $n \times 1$ вместо $n \times n$; ровно это делает Inception в поздних версиях.

2.5.2. Сгруппированная свёртка (group convolution)

Каналы входа режутся на g групп, и каждая группа сворачивается своим независимым набором фильтров; выходы конкатенируются. Взаимодействий «каждый канал с каждым фильтром» больше нет, параметров и операций в g раз меньше: $h_K w_K \times C \times F/g$.

У приёма забавное происхождение: это не оптимизация, а вынужденная мера из AlexNet. Сеть не помещалась в память одной GPU, её разрезали на две половины, которые почти не обменивались данными, фактически свёртки с $g = 2$. Позже идею сделали осознанной: на ней построен ResNeXt из следующей лекции.

2.5.3. Depthwise и depthwise separable

Предельный случай $g = C$: каждый канал сворачивается собственным ядром $k \times k$, каналы вообще не смешиваются — это *depthwise*-свёртка, всего $k^2 \times C$ параметров. Смешивание каналов возвращают отдельным дешёвым шагом — уже знакомой поточечной свёрткой 1×1 . Пара «depthwise + pointwise» называется *depthwise separable* свёрткой:

$$\underbrace{k^2 C}_{\text{Depthwise}} + \underbrace{C \times F}_{\text{Pointwise}} \text{ против } k^2 \times C \times F, \quad \frac{k^2 C + CF}{k^2 CF} = \frac{1}{F} + \frac{1}{k^2}.$$

При $k = 3$ и сотнях фильтров это выигрыш почти на порядок ценой небольшой потери качества.

На свёртках depthwise separable построены MobileNet и Xception, к ним вернёмся в следующей лекции.

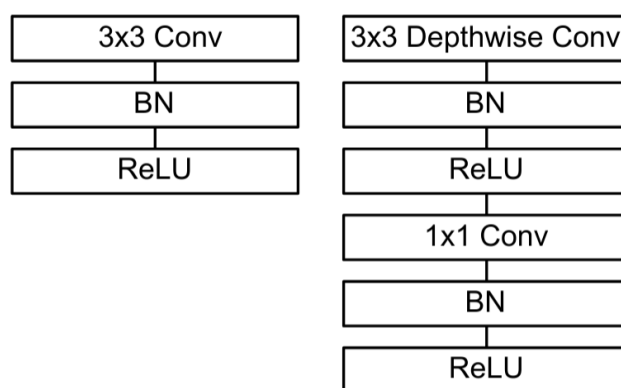


Figure 3. Left: Standard convolutional layer with batchnorm and ReLU. Right: Depthwise Separable convolutions with Depthwise and Pointwise layers followed by batchnorm and ReLU.

Рисунок 7. Слой MobileNet (Howard и соавторы, 2017): слева обычная свёртка 3×3 с BN и ReLU, справа она же, разложенная на depthwise 3×3 и pointwise 1×1

ПОДСКАЗКА

Как это удобно запомнить

Обычная свёртка смешивает информацию сразу по двум измерениям: по пространству (окно $k \times k$) и каналам (вся глубина C). Все три приёма выше — разные способы **не делать этого одновременно**: separable разделяет пространственные оси, group ограничивает смешивание каналов группами, depthwise separable разводит пространство и каналы по двум отдельным шагам.

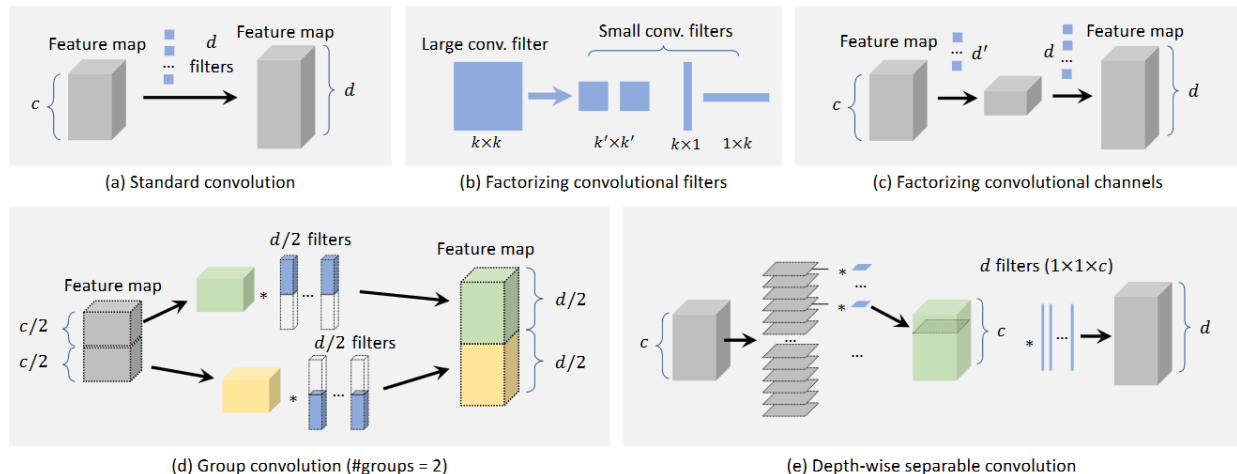


Fig. 10: An overview of speed up methods of a CNN's convolutional layer and the comparison of their computational complexity: (a) Standard convolution: $\mathcal{O}(dk^2c)$. (b) Factoring convolutional filters ($k \times k \rightarrow (k' \times k')^2$ or $1 \times k, k \times 1$): $\mathcal{O}(dk'^2c)$ or $\mathcal{O}(dkc)$. (c) Factoring convolutional channels: $\mathcal{O}(d'k^2c) + \mathcal{O}(dk^2d')$. (d) Group convolution (#groups= m): $\mathcal{O}(dk^2c/m)$. (e) Depth-wise separable convolution: $\mathcal{O}(ck^2) + \mathcal{O}(dc)$.

Рисунок 8. Сводка способов удешевить свёрточный слой (обзор Zou и соавторов, 2019): (a) обычная свёртка, (b) факторизация ядра, (c) факторизация каналов, (d) group convolution, (e) depthwise separable с оценками сложности

2.6. ТРАНСПОНИРОВАННАЯ СВЁРТКА

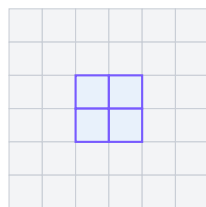
Все рассмотренные свёртки либо сохраняют пространственный размер, либо уменьшают его. Для классификации этого достаточно: большая картинка постепенно ужимается до вектора признаков, поверх которого стоит классификатор.

Но на практике часто нужен обратный процесс: из маленького представления вернуться к картинке в полном разрешении — в сегментации, автокодировщиках, генеративных моделях. Нужна операция, увеличивающая размер выхода по сравнению с входом. Это транспонированная свёртка (*transposed convolution*, иногда *deconvolution*, хотя название неудачное).

Устроена она просто: вход обкладывается нулями (*padding*, вообще говоря, большой), после чего применяется обычная свёртка. Изображение 2×2 с $\text{padding} = 2$ превращается в 6×6 , свёртка ядром 3×3 даёт 4×4 — размер вырос. Если у прямой свёртки был $\text{stride} = 2$, то у транспонированной нули вставляются ещё и между элементами входа.

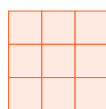
Вход 2×2 , обложен нулями

Серое — нули



*

Ядро 3×3



=

Выход 4×4

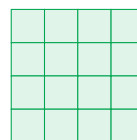


Рисунок 9. Транспонированная свёртка: размер выхода больше размера входа

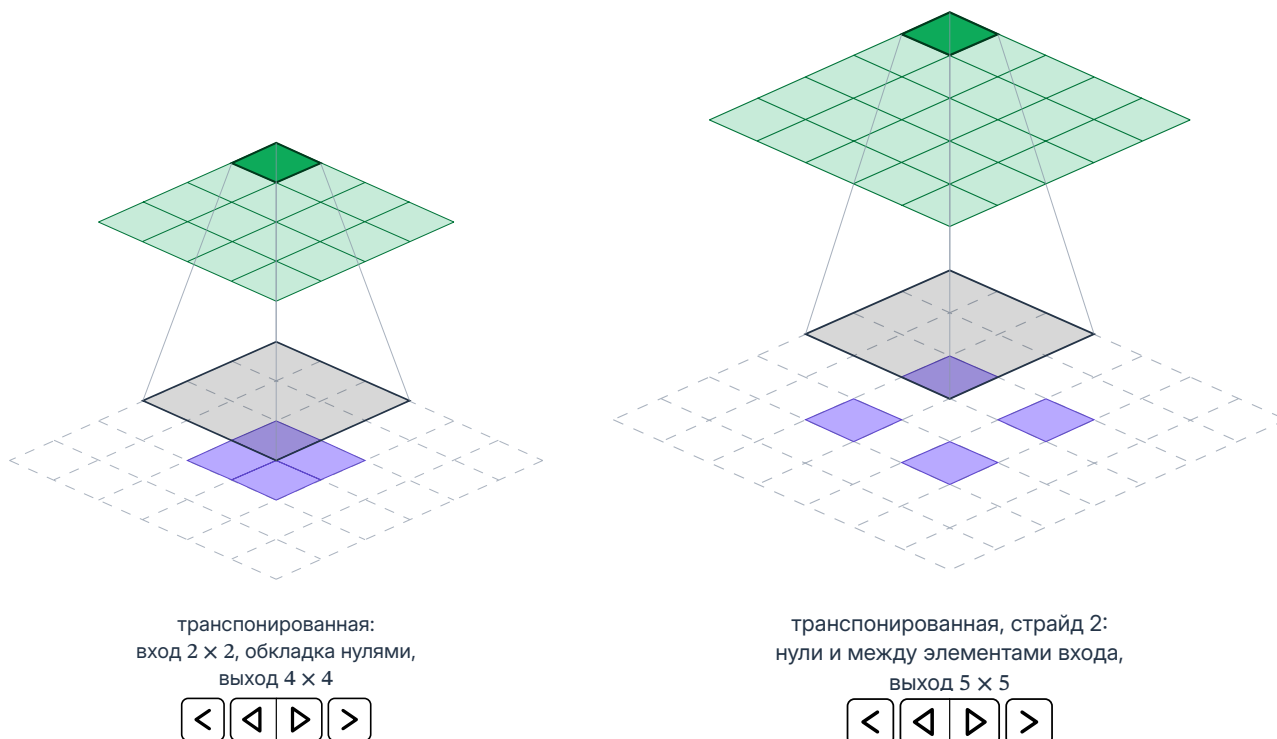


Рисунок 10. Анимация (Acrobat) нарисована векторно: транспонированная свёртка без страйда и со страйдом 2.

Синие клетки — элементы входа 2×2 , пунктирные — добавленные нули; при страйде 2 нули вставлены и между элементами входа. Выход получается больше входа

2.6.1. Почему именно так и почему транспонированная

Величина нулевой обкладки определяется требованием сохранить *поле зависимости* (receptive field, рецептивное поле). На прямом проходе каждый пиксель выхода зависит от вполне определённого окна во входе: при ядре 2×2 — от четырёх пикселей. Посмотрим на ту же связь с другой стороны: возьмём пиксель входа и спросим, на какие пиксели выхода он влияет. Угловой пиксель влияет ровно на один выход, центральный — на все четыре.

Транспонированная свёртка строится так, чтобы **картина связей сохранилась, но роли входа и выхода поменялись местами**. Если на прямом проходе выход y_1 зависел от входов x_1, x_2, x_4, x_5 , то в транспонированной операции x_1, x_2, x_4, x_5 должны зависеть от y_1 . Это обеспечивает нулевая обкладка нужного размера. При $\text{stride} = 2$ на прямом проходе позиции ядра были прорежены, поэтому в обратной операции надо прореживать вход, отсюда — нули между элементами. Название становится понятным в следующем разделе: свёртка — умножение на матрицу $\mathcal{K}$, описанная операция — умножение на $\mathcal{K}^T$.

3. Обратный проход через свёрточный слой

С прямым проходом разобрались. Теперь надо понять, как через свёртку проходит градиент: зная $\partial L / \partial Y$, вычислить $\partial L / \partial X$, чтобы отдать градиент дальше вниз, и $\partial L / \partial K$ (чтобы обновить сам фильтр).

Разберём на маленьком примере, где всё выписывается явно.

3.1. СВЁРТКА — ЭТО ЛИНЕЙНАЯ ОПЕРАЦИЯ

Пусть вход X — изображение 3×3 , ядро K — размера 2×2 . Позиций для ядра четыре, значит, выход Y имеет размер 2×2 :

$$X = \begin{pmatrix} x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 \\ x_7 & x_8 & x_9 \end{pmatrix}, \quad K = \begin{pmatrix} k_1 & k_2 \\ k_3 & k_4 \end{pmatrix}, \quad Y = \begin{pmatrix} y_1 & y_2 \\ y_3 & y_4 \end{pmatrix}.$$

Распишем выход по определению:

$$\begin{aligned} y_1 &= k_1 x_1 + k_2 x_2 + k_3 x_4 + k_4 x_5, & y_2 &= k_1 x_2 + k_2 x_3 + k_3 x_5 + k_4 x_6, \\ y_3 &= k_1 x_4 + k_2 x_5 + k_3 x_7 + k_4 x_8, & y_4 &= k_1 x_5 + k_2 x_6 + k_3 x_8 + k_4 x_9. \end{aligned}$$

Каждый выход — линейная комбинация входов. Значит, если вытянуть X в вектор $x \in \mathbb{R}^9$, а Y — в вектор $y \in \mathbb{R}^4$, свёртку можно записать одним матричным умножением:

$$y = \mathcal{K}x, \quad \mathcal{K} = \begin{pmatrix} k_1 & k_2 & 0 & k_3 & k_4 & 0 & 0 & 0 & 0 \\ 0 & k_1 & k_2 & 0 & k_3 & k_4 & 0 & 0 & 0 \\ 0 & 0 & 0 & k_1 & k_2 & 0 & k_3 & k_4 & 0 \\ 0 & 0 & 0 & 0 & k_1 & k_2 & 0 & k_3 & k_4 \end{pmatrix}. \quad (4)$$

Матрица $\mathcal{K}$ имеет размер 4×9 и очень специальную структуру: в ней всего четыре независимых числа, каждое повторяется в нескольких позициях, всё остальное — нули. Такие матрицы называют *тёплицевыми* (точнее, *блочно-тёплицевыми*).

Соответствие «позиция окна $\leftrightarrow$ строка матрицы» удобно увидеть в движении.

Окно ядра для y_1

x_1	x_2	x_3
x_4	x_5	x_6
x_7	x_8	x_9

$$y_1 = k_1 x_1 + k_2 x_2 + k_3 x_4 + k_4 x_5 : \quad \text{строка } (k_1 \ k_2 \ 0 \ k_3 \ k_4 \ 0 \ 0 \ 0 \ 0)$$

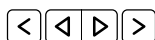


Рисунок 11. Анимация (Acrobat): позиция окна ядра и соответствующая ей строка матрицы $\mathcal{K}$. Кнопкой «шаг» удобно проходить строки по одной

**Практическая польза: батч одним умножением**

Это представление не только инструмент вывода. Если нужно применить свёртку с фиксированным ядром не к одной картинке, а к батчу из сотен картинок, наивная реализация — цикл по объектам с вызовом свёртки внутри. Гораздо эффективнее собрать батч в матрицу (по столбцу на объект) и один раз умножить матрицу на матрицу. Одна операция вместо цикла, и она уже оптимизирована в BLAS.

Этот приём называется *im2col*: развернуть все окна изображения в столбцы и свести свёртку к плотному матричному умножению (GEMM).

3.2. ГРАДИЕНТЫ В МАТРИЧНОЙ ФОРМЕ

Дифференцируем $y = \mathcal{K}x$ тем же приёмом с дифференциалами, каким в [лекции 2](#) выводился backpropagation; вся разница в том, что теперь матрица не произвольная, а собрана из четырёх чисел ядра:

$$dY = d\mathcal{K} \cdot x + \mathcal{K} \cdot dx.$$

Отсюда сразу получаются оба нужных градиента. Из второго слагаемого:

$$\frac{\partial L}{\partial x} = \mathcal{K}^\top \frac{\partial L}{\partial y}, \quad (5)$$

а из первого, если на секунду считать элементы $\mathcal{K}$ независимыми:

$$\frac{\partial L}{\partial \mathcal{K}} = \frac{\partial L}{\partial y} x^\top \quad \text{— матрица ранга 1.} \quad (6)$$

Теперь надо обе формулы превратить обратно в свёртки. Это важно: свёртка — операция, для которой существуют высокооптимизированные реализации. Если обратный проход выражается через неё же, мы получаем его бесплатно.

1. Прямой проход, матрично: $y = \mathcal{K}x$.

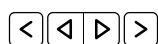


Рисунок 12. Вывод по шагам (Acrobat): кнопка «шаг» открывает строки по одной — удобно для доски и самопроверки

3.3. ГРАДИЕНТ ПО ВХОДУ — ЭТО СВЁРТКА С РАЗВЁРНУТЫМ ЯДРОМ

Возьмём формулу (5). Обозначим карту градиентов на выходе через $G = \partial L / \partial y$ — это изображение 2×2 с элементами $g_1, \dots, g_4$.

УТВЕРЖДЕНИЕ

Обложим G рамкой нулей до размера 4×4 и свернём с ядром, в котором элементы K поставлены в **обратном порядке**:

$$\tilde{K} = \begin{pmatrix} k_4 & k_3 \\ k_2 & k_1 \end{pmatrix}.$$

Изображение 4×4 с ядром 2×2 даёт выход 3×3 . Это в точности $\partial L / \partial X$.

Проверим на двух элементах.

Угловой. Из (5) $\partial L / \partial x_1$ это первый столбец $\mathcal{K}$, скалярно умноженный на G . Первый столбец равен $(k_1, 0, 0, 0)^T$, поэтому

$$\frac{\partial L}{\partial x_1} = k_1 g_1.$$

Теперь то же из свёртки. В первой позиции окно 2×2 покрывает левый верхний угол обложенной картинки: три нуля и g_1 . Ядро $\tilde{K}$ даёт $k_4 \times 0 + k_3 \times 0 + k_2 \times 0 + k_1 g_1 = k_1 g_1$. Совпало, и заметь, сработало именно то, что элементы ядра развёрнуты: с g_1 встретился k_1 , а не k_4 .

Центральный. Пятый столбец $\mathcal{K}$ равен $(k_4, k_3, k_2, k_1)^T$, значит,

$$\frac{\partial L}{\partial x_5} = k_4 g_1 + k_3 g_2 + k_2 g_3 + k_1 g_4.$$

Свёртка в центральной позиции покрывает все четыре элемента G и, поскольку ядро развёрнуто, умножает их ровно на те же коэффициенты. Совпало.



Итог

Прямой проход: свёртка с ядром K .

Обратный проход по входу: свёртка карты градиентов, обложенной нулями, с ядром K , развёрнутым на 180° .

Это ровно та операция, которую мы в предыдущем разделе называли транспонированной свёрткой, потому что она и есть умножение на $\mathcal{K}^T$.

3.4. ГРАДИЕНТ ПО ФИЛЬТРУ — ЭТО СВЁРТКА ВХОДА С КАРТОЙ ГРАДИЕНТОВ

С (6) есть тонкость. Матрица $\mathcal{K}$ не произвольная: в ней четыре независимых параметра, каждый сидит в нескольких позициях, а остальные элементы жёстко равны нулю и не меняются. Формула (6) даёт градиент так, как если бы все 36 элементов были свободны.

С точки зрения графа вычислений картина такая: параметр k_1 порождает элементы матрицы в позициях $(1, 1)$, $(2, 2)$, $(3, 4)$, $(4, 5)$. Это тождественное преобразование, оно градиент не меняет. Дальше эти элементы участвуют в вычислении y . Значит, чтобы получить градиент по k_1 , надо **просуммировать** градиенты по всем позициям, куда k_1 попал:

$$\frac{\partial L}{\partial k_1} = \frac{\partial L}{\partial \mathcal{K}_{11}} + \frac{\partial L}{\partial \mathcal{K}_{22}} + \frac{\partial L}{\partial \mathcal{K}_{34}} + \frac{\partial L}{\partial \mathcal{K}_{45}}.$$

Каждое слагаемое берётся из матрицы ранга 1 в (6), то есть равно произведению соответствующих компонент G и x :

$$\frac{\partial L}{\partial k_1} = g_1 x_1 + g_2 x_2 + g_3 x_4 + g_4 x_5. \quad (7)$$

А теперь посмотрим на (7) внимательно. Это скалярное произведение окна 2×2 изображения X с верхним левым углом в позиции $(1, 1)$ на карту G . То есть

УТВЕРЖДЕНИЕ

$$\frac{\partial L}{\partial K} = X * G.$$

Градиент по фильтру — это свёртка **входного изображения** 3×3 с картой градиентов G размера 2×2 в роли ядра. Выход как раз 2×2 — размер фильтра.

Аналогично проверяется k_2 : он сидит в позициях $(1, 2)$, $(2, 3)$, $(3, 5)$, $(4, 6)$, что даёт $g_1 x_2 + g_2 x_3 + g_3 x_5 + g_4 x_6$, то же скалярное произведение, но окно сдвинуто на один пиксель вправо. Ровно вторая позиция свёртки.

Что считаем	Как
Y (прямой проход)	Свёртка X с ядром K
$\partial L / \partial X$	Свёртка G , обложенной нулями, с ядром K , развёрнутым на 180°
$\partial L / \partial K$	Свёртка X с ядром G

Таблица 1. Прямой и обратный проход свёрточного слоя



Почему всё это имело смысл выводить

Реализовать свёртку можно множеством способов: развернуть в матричное умножение (im2col + GEMM), перейти к образам Фурье, перемножить и вернуться обратно, использовать алгоритм Винограда для маленьких ядер. У NVIDIA есть собственные реализации в cuDNN, подобранные под архитектуру конкретной GPU так, чтобы максимально распараллелить вычисление.

Иными словами, свёртка — это операция, которую производитель библиотеки или оборудования предоставляет уже эффективно реализованной. Поэтому и стоило потрудиться, чтобы выразить обе формулы обратного прохода тоже через свёртку: как только это сделано, весь backward считается теми же оптимизированными ядрами, что и forward, и ничего дополнительно писать не нужно.

У этой абстракции есть и обратная сторона — **воспроизводимость**. Какой алгоритм выберет библиотека, зависит от версии, железа и даже размеров тензоров, а разные алгоритмы дают чуть разные ошибки округления. Поэтому один и тот же код на разных машинах может выдавать немного разные числа; для строгой воспроизводимости в PyTorch есть детерминированные режимы, за которые приходится платить скоростью.

4. Пулинг

4.1. ЧТО ЭТО

Пулинг — операция уменьшения пространственного размера без обучаемых параметров. Изображение разбивается на окна (обычно 2×2 со stride 2), и из каждого окна берётся одно число:

- **max pooling** — максимум по окну;
- **average pooling** — среднее по окну.

Размер окна и шаг настраиваются так же, как у свёртки. Важно: пулинг применяется к каждому каналу независимо, число каналов не меняется.

4.2. ЗАЧЕМ: ИНВАРИАНТНОСТЬ К МАЛЫМ СДВИГАМ

Возьмём изображение с каким-то объектом, прогоним через свёрточный слой, потом через max pooling 2×2 . Теперь сдвинем объект на один пиксель вправо и повторим.

Свёртка — операция, коммутирующая со сдвигом: все активации просто сдвинутся на тот же пиксель. А когда мы берём максимум по окну 2×2 , то самая сильная активация с большой вероятностью останется внутри того же окна и максимум не изменится.

Получается, что max pooling даёт **частичную инвариантность к небольшим сдвигам объекта во входном изображении**. Для классификации это ровно то, что нужно: метка класса не должна зависеть от того, стоит объект на два пикселя левее или правее.

4.3. КАК ЧЕРЕЗ ПУЛИНГ ИДЁТ ГРАДИЕНТ

Максимум — функция недифференцируемая; строго говоря, у неё есть только субдифференциал. Но на практике всё просто. На прямом проходе мы запоминаем, *из какой позиции* окна взят максимум. На обратном проходе пришедший градиент целиком отправляется в эту позицию, а всем остальным элементам окна ставится ноль.

$$\frac{\partial L}{\partial x_{ij}} = \begin{cases} \frac{\partial L}{\partial y_{\text{окна}}}, & (i, j) = \arg \max \text{ по окну,} \\ 0, & \text{иначе.} \end{cases}$$

Для average pooling всё ещё проще: градиент делится поровну между всеми элементами окна.

ЗАМЕЧАНИЕ

Пулинг или свёртка с шагом?

Уменьшать пространственный размер можно двумя способами: пулингом или свёрткой со stride 2. Пулинг не имеет параметров и даёт инвариантность к малым сдвигам; свёртка с шагом — это *обучаемое сжатие*: сеть сама решает, что сохранить при уменьшении размера. Обе стратегии живы, и в следующей лекции это будет видно на классических сетях: VGG уменьшает размер пулингом, ResNet внутри стадий — страйдом в первой свёртке блока, часто у сети остаётся один-два пулинга в начале и global average pooling в конце.

5. Рецептивное поле

Рецептивное поле (receptive field) элемента — это множество пикселей входного изображения, от которых он зависит. Понятие простое, но именно оно объясняет большую часть архитектурных решений, о которых пойдёт речь дальше.

Ключевое наблюдение: рецептивное поле растёт с глубиной. Рассмотрим каскад из двух свёрток 3×3 .

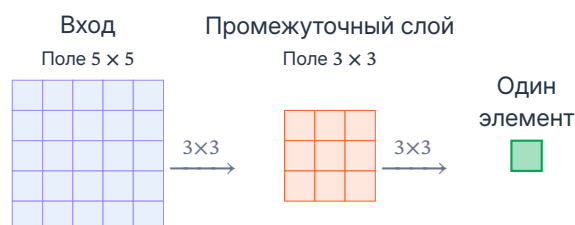


Рисунок 13. Каскад из двух свёрток 3×3 даёт такое же рецептивное поле, как одна свёртка 5×5 , но дешевле

Один элемент выхода зависит от окна 3×3 промежуточного слоя. Каждый из этих девяти элементов, в свою очередь, зависит от окна 3×3 входа. Объединение этих окон — квадрат 5×5 .

Значит, два слоя 3×3 дают то же рецептивное поле, что один слой 5×5 . Сравним цену (на пару каналов):

$$\underbrace{3^2 + 3^2 = 18}_{\text{Два слоя } 3 \times 3} \quad \text{против} \quad \underbrace{5^2 = 25}_{\text{Один слой } 5 \times 5}.$$

Параметров меньше, а вычислений — тем более: сложность свёртки растёт как квадрат размера ядра, поэтому переход с 3×3 на 5×5 делает дороже операцию не в два раза, а почти в три. Плюс между двумя слоями стоит нелинейность, то есть каскад ещё и выразительнее.

ЗАМЕЧАНИЕ

Это работает и дальше

Три свёртки 3×3 дают рецептивное поле 7×7 ценой 27 параметров вместо 49. Именно на этом наблюдении построена архитектура VGG, а позже — разложение свёрток в Inception; обе — в следующей лекции.

Второй способ расширить рецептивное поле, не увеличивая число параметров, — это уже знакомый dilation.

6. Dropout и Batch Normalization в свёрточных слоях

Оба приёма разобраны в [лекции 3](#), но там речь шла о полносвязных сетях. В свёрточных слоях у обоих есть специфика, и её стоит знать.

Dropout. Обычный Dropout зануляет отдельные активации. В свёрточном слое это почти не работает: соседние элементы карты признаков сильно коррелированы, сеть спокойно восстанавливает занулённый пиксель по его соседям — регуляризующий эффект исчезает. Поэтому для свёрток используют *SpatialDropout* (в PyTorch — `nn.Dropout2d`): зануляются не отдельные значения, а **целые каналы** целиком. Убрать канал по соседям уже не получится. Той же логике подчиняется аугментация Cutout —

вырезание случайного прямоугольника из входного изображения. По сути, это Dropout, применённый прямо к входу. На практике, впрочем, в свёрточной части сетей Dropout ставят редко: с переобучением там справляются нормализация и аугментации, а Dropout остаётся в полносвязной голове, как в AlexNet и VGG.

Batch Normalization. В полносвязном слое BN нормирует каждый нейрон отдельно по статистикам батча. В свёрточном слое так делать нельзя: у слоя нет «отдельных нейронов», у него есть каналы, и одно и то же ядро применяется во всех позициях. Поэтому BN в свёртках нормирует **каждый канал целиком**: среднее и дисперсия считаются сразу по батчу и по обеим пространственным осям — по $N \times H \times W$ значениям, на канал приходится одна пара обучаемых параметров (γ, β) . Это сохраняет главное свойство свёртки — независимость от позиции: нормировка одинакова во всех точках карты.

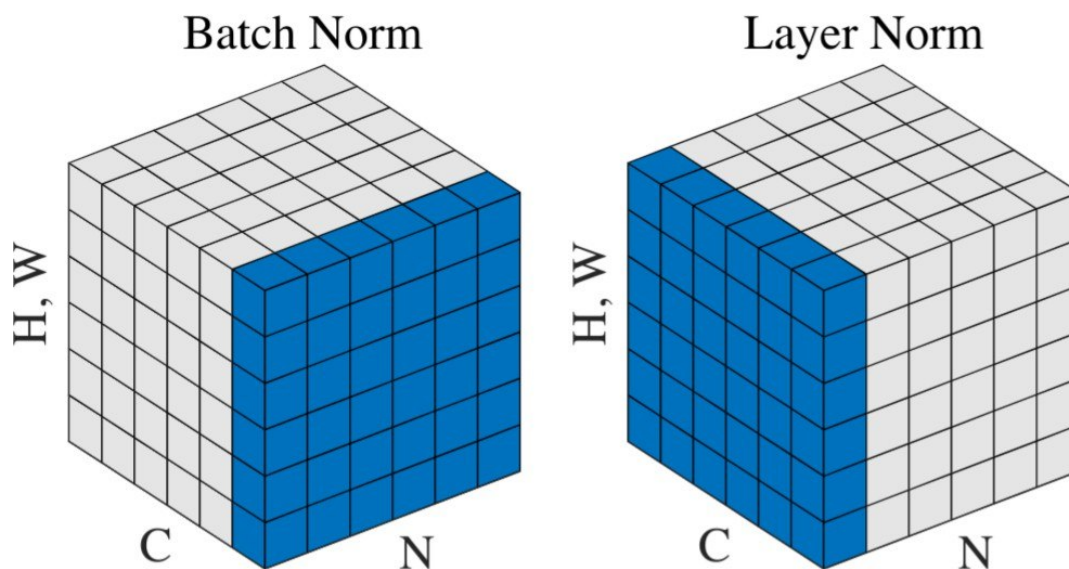


Рисунок 14. По каким осям считаются статистики (синее): BatchNorm — по батчу и пространству для каждого канала, LayerNorm — по каналам и пространству для каждого объекта. LayerNorm ещё встретится в ConvNeXt и трансформерах

7. Итоги

Главное из лекции

- **Зачем свёртки.** Полносвязный слой на изображении требует непоправимого числа параметров. Локальность связей и разделение весов сокращают его на много порядков.
- **Геометрия.** Размер выхода полностью определяется формулой (2) через padding, stride и dilation. По каналам ядро всегда покрывает вход целиком, потому что между каналами нет отношения соседства. Число фильтров задаёт число каналов на выходе.
- **Обратный проход.** Свёртка — линейная операция, $y = \mathcal{K}x$. Отсюда $\partial L / \partial X$ — свёртка обложенной нулями карты градиентов с развёрнутым ядром (она же транспонированная свёртка), а $\partial L / \partial K$ — свёртка входа с картой градиентов в роли ядра. Обе операции — это снова свёртки, поэтому используют те же оптимизированные реализации.
- **Пулинг.** Уменьшает размер, даёт частичную инвариантность к сдвигам, обучаемых параметров не имеет. Градиент max pooling идёт только в позицию максимума.

- **Рецептивное поле.** Каскад мелких свёрток даёт то же поле, что одна крупная, но дешевле.
- **Экономия параметров.** Полную свёртку можно разбивать на множители: по пространственным осям (separable), группам каналов (group) или с полным разделением пространства и каналов (depthwise separable, выигрыш $\approx 1/F + 1/k^2$).
- **Регуляризация и нормализация.** В свёрточных слоях Dropout зануляет каналы (Dropout2d), а BatchNorm нормирует каждый канал по батчу и обоим пространственным осям.



Что почитать

Dumoulin V. *A guide to convolution arithmetic for deep learning* — наглядный разбор всех режимов свёртки; отсюда взяты анимации, использованные в этом конспекте.

Теперь есть все кирпичи: свёрточный слой со всеми его вариантами, пулинг и понимание того, как через них идёт градиент. В следующей лекции из этих кирпичей будут собираться настоящие сети — от LeNet 1998 года до ConvNeXt — и станет видно, какая инженерная идея добавлялась на каждом шаге.